

ambire-common / contracts/WALLET

Security Review

WALLET token — `WALLET.sol`

files reviewed	commit	passes	findings / leads
1	63d08560c	3	2 / 6

01

Summary

2	1	6	3
findings	below threshold	leads	passes

02

Scope

Mode	filename
Files reviewed	`contracts/WALLET/WALLET.sol`
Passes	3
Memory	0 records before this scan · 9 after · `63d08560c`

03

Findings List

#	confidence	title
01	85	A vesting mint after the end blocks every later xWALLET enter
02	75	The supply cap check stops a mint that reaches MAX_SUPPLY exactly
Below Confidence Threshold		
03	70	A vesting mint above the cap headroom blocks every xWALLET enter

01

confidence

85

seen in 2/3 runs

NEW

A vesting mint after the end blocks every later xWALLET enter

`WALLETSupplyController.mintableVesting`

Description

After `end`, the first mintVesting stores a time past `end`, so each later mintableVesting reverts with VESTING_OVER and every xWALLET enter reverts until governance changes the vesting.

Fix

```
        if (block.timestamp > end) {
-           require(end > lastMinted, "VESTING_OVER");
+           if (lastMinted >= end) return 0;
            return (end - lastMinted) * amountPerSecond;
```

02

confidence

75

seen in 3/3 runs

NEW

The supply cap check stops a mint that reaches MAX_SUPPLY exactly

`WALLETToken.innerMint`

Description

The innerMint check uses `<` and not `<=`, so the supplyController can never mint the last wei of MAX_SUPPLY, and a mint to exactly the cap reverts.

Fix

```
-       require(totalSupply < MAX_SUPPLY, 'MAX_SUPPLY');
+       require(totalSupply <= MAX_SUPPLY, 'MAX_SUPPLY');
```

03

confidence

70

seen in 3/3 runs

NEW

below confidence threshold

A vesting mint above the cap headroom blocks every xWALLET enter

`WALLETToken.innerMint`

Description

When the accrued vesting is more than the cap headroom, innerMint reverts the full mint, so every xWALLET.enter, xWALLET.enterTo and staked claim fails until governance acts.

04
Leads

> *Vulnerability trails where the full exploit path could not be completed in one analysis pass. These are not false positives — they are high-signal leads for manual review. Not scored.*

L01 Any caller can set lastRoot back to an old signed root

`WALLETSupplyController.claimWithRootUpdate` seen in 1/3 runs NEW

the governance signature covers only newRoot, with no nonce, no chain ID and no check that the root is newer.

> An attacker can resubmit an old root, so claims under the newer tree revert until someone submits the newer root again. We found no loss of tokens.

L02 Burned penalty tokens stay in totalSupply and use the cap

`WALLETToken.innerMint` seen in 3/3 runs NEW

the token has no burn function, and SupplyController.claim burns the penalty with WALLET.mint(address(0), toBurn).

> Each claim with a penalty uses MAX_SUPPLY headroom for tokens that nobody can spend. We did not confirm whether the team intends this.

L03 xWALLET shareValue counts vesting that innerMint rejects

`WALLETToken.innerMint` seen in 2/3 runs NEW

StakingPool.shareValue adds mintableVesting to the balance with no cap limit, but innerMint reverts that mint at the cap.

> An integrator reads a share value that no user can redeem. We found no on-chain caller that trusts shareValue.

L04 The permit domain separator keeps the chain ID from deployment

`WALLETToken.permit` seen in 3/3 runs NEW

the constructor computes DOMAIN_SEPARATOR once with chainid(), and permit never compares it with block.chainid.

> After a chain fork, an attacker can use one permit signature on both chains. We did not show a fork of the deployment chain.

L05 An attacker can submit a permit signature first

`WALLETToken.permit` seen in 3/3 runs NEW

permit increments the nonce, and any caller can submit a signature from the mempool.

> An external contract that calls permit and then transferFrom without try/catch reverts. No caller in this repository uses WALLET.permit.

L06 **permit accepts only ECDSA signatures from the owner**

`WALLETToken.permit` seen in 2/3 runs NEW

permit compares the ecrecover result with owner and has no EIP-1271 isValidSignature path.

> Owners that are smart contract accounts cannot use permit. We did not confirm that an Ambire flow needs permit for smart accounts.

Disclaimer

This review was performed by an AI assistant. AI analysis can never verify the complete absence of vulnerabilities and no guarantee of security is given. Team security reviews, bug bounty programs, and on-chain monitoring are strongly recommended. For a consultation regarding your projects' security, visit https://www.pashov.com